STAGE 2 – PRACTICAL ENGINEERING CHALLENGE

TEAM ID: LRC-26-0388	TEAM NAME: Finite Element Gang.

Problem Statement: Industrial Manufacturing Process Automations

Industrial manufacturing depends on highly controlled liquid-handling systems to deliver process fluids reliably from a source or bulk supply to downstream process equipment. Depending on the application, these fluids may include **ultrapure fluids, process chemicals, solvents, developers, or CMP-related fluids**. Industry standards explicitly address liquid-chemical distribution, high-purity piping, bulk chemical distribution, system qualification, and UPW system operation and monitoring.

In such systems, **flow is a fundamental process variable**. An incorrect flow rate can indicate or cause a restriction, incorrect valve state, inadequate pumping performance, abnormal process demand, or another hydraulic deviation. The consequence may be unstable process conditions, incorrect chemical delivery, equipment malfunction, or loss of process performance.

This challenge therefore focuses on a fundamental industrial facilities and process-engineering task: **To establish a defined process-fluid flow, measure it reliably, detect a deviation, diagnose its cause, and restore the process to the required operating condition.**

Engineering Challenge

As you are part of a multidisciplinary engineering team responsible for designing, building and validating a small-scale automated liquid-flow system represented of industrial process handling system.

CASE 1 – CONTROL + MEASUREMNT

Problem: Develop a system that manages the changing level of fluid in the sump automatically (when there is a sudden external fluid charging to the sump) and while accurately measuring and totalizing the fluid discharged from the sump using the flow transmitter.

CASE 2 – FLUSHING +SHUTDOWN

Problem: Develop an alternative operating configuration that allows flush water to pass through the system while all three pumps operate simultaneously, and then safely transition the system to an OFF condition when the flush-water supply has been exhausted.

Given Process Parameters

Pipe diameter =	**8mm**
Fluid density (ρ) =	**1000kg/m^3**
Dynamic Viscosity (μ) =	**1x10^{-3}Pa.s**
Required Flow range =	**1-6 L/min**
Nominal target flow for the live challenge =	**3L/min**

Available Hardware

Mechanical Hardware	Electronic Hardware
1. Water Tank – 3 Quantity 2. Pumps – 3 Quantity 3. FT-01 Flow Transmitter – 01 Quantity 4. Ball valve – 3 Quantity 5. ON/OFF/XV Valve – 1 Quantity 6. 10-mm OD tubing/blue – 9 meters/1 meter 7. Microcontroller/PCB/ data-acquisition software 8. Required electrical/ electronic accessories	1. Custom PCB 2. ESP32 NodeMCU module 3. Pump Driver-1 4. Pump Driver-2 5. Pump Driver-3 6. 12 V DC power input (**SMPS**) 7. 12 VDC Solenoid-valve interface 8. Flow-sensor interface 9. Float-switch interface 10. ON/OFF-switch interface 11. Pump-driver PWM/enable interfaces 12. On/Off Switch

The team must determine the appropriate arrangement, connections and measurement method.

- Assemble and commission the liquid-flow system using the components provided.
- Measure and characterize the flow using FT-01
- Interface FT-01 with the provided **data-acquisition system**
- Experimentally determine the relationship between sensor output and actual flow.
- Characterize the effect of valve position on flow rate. Record and analyse flow vs time data
- Maintain target flow of 3L/min
- Use graphs and measured data to justify engineering decisions
- Design an appropriate hydraulic arrangement and P&ID

TEAM INSTRUCTIONS

- As a team go through the list provided to read about each component that has been Listed.
- The PCB is a 12 V DC and the SMPS connections should be made carefully.

HYDRAULICS SYSTEM DESIGN

Q1.1 Design and build the hydraulic path with the flow transmitter on the recirculation line. Submit/sketch a labelled P&ID below by showing the pumps, Needle valves, flow transmitter, flow direction, Tank, sump, tube diameter and measurement points.

CASE 1 – Process Fluid Transfer & Automatic Level Protection

Objective: Set-up and operate a closed-loop process- fluid handling system using the available tank, pump, valves flow transmitter, sump, float sensor and control PCB. The System must continuously transfer process fluid recirculation system by connecting the 10L fluid tank with a On/Off valve to a pump-01 and the pump-01 goes to the inlet of sump 4.2L tank. The Sump 4.2L outlet line is connected to another pump-02 followed by a flow transmitter that measures the total flow back to the 10L fluid tank. The Pump-01 runs on 50% duty cycle and Pump-02 runs on 100% duty cycle. **Calibrate the flow transmitter** and demonstrate correct fluid circulation, if there is sudden introduction of external fluid into the Sump tank then Pump-01 should stop while pump – 02 should still runs till fluid level reduces in the sump. After the fluid reduction in the SUMP 4.2L tank the Pump-01 should turn on and the process must continue. **Also, demonstrate Flow vs Time trends in mL/s, total transferred Liters back to the 10L tank** (share Screenshots flow in L/min). **Share the code.** The case-1 can be turned on using the available on/off Switch, to start the system.

CASE 2 – Flush/Line Conditioning Sequence

Objective: After successfully completing case 1, initiate the flush water in the sequence using the **equal Tee.** The On/Off valve must be switched off in order to isolate the 10L process fluid tank. The flush Fluid tank (4.2L) is used as a temporary fluid source to flush out the process fluid or any debris from the system. Before the start of this process make sure you open the manual ball valve on the fluid flush line before Pump-03 starts. The entire process of case -1 and case - 2 should run in a single go without manual intervention except opening of ball valve before the Pump-03 starts. **The System must stop as soon as the tank flush fluid 4.2L tank finishes.** The system must be automated without human interventions (only ball valve can be opened manually before the start of flush line sequence)

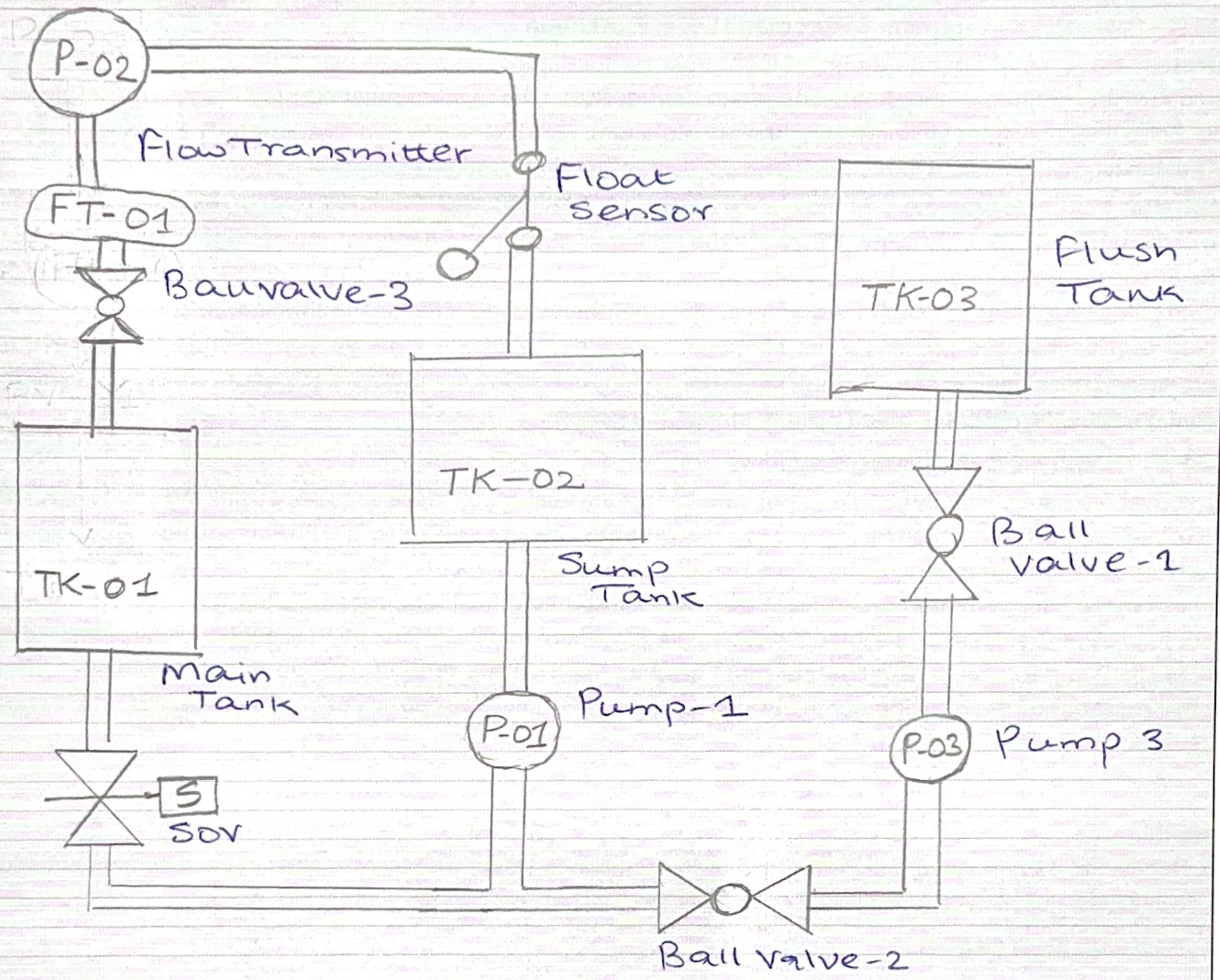
SUBMIT:

A. Submit: 10-15 min Video and Photograph of the completed setup, with identification of each component and explain the direction of flow in the video verbally. Demonstrate both Case - 1 and Case – 2.

B. Final P&ID Block diagram

Pump-2

P&ID SKETCH / WORKING SPACE



Q1.2 After setting-up the system, now operate it at approximately at 1L/min, 3L/min & 6L/min. Compare the theoretical velocity with the actual operating behaviour.

WORKING / COMPARISON

Theoretical Flow Velocity:

$Q = AV \Rightarrow V = \frac{Q}{A}$; $A = \frac{\pi d^2}{4}$; $Re = \frac{\rho V D}{\mu}$; $A = \frac{\pi \times 0.008^2}{4} = 5.027 \times 10^{-5} m^2$

$Q_1 = \frac{1 \times 10^{-3}}{60} = 1.667 \times 10^{-5} m^3/s$

$V_1 = \frac{1.667 \times 10^{-5}}{5.027 \times 10^{-5}} = 0.332 m/s$

$Re_1 = \frac{1000 \times 0.332 \times 0.008}{1 \times 10^{-3}} = 2656$

$Q_2 = \frac{3 \times 10^{-3}}{60} = 5 \times 10^{-5} m^3/s$

$V_2 = \frac{5 \times 10^{-5}}{5.027 \times 10^{-5}} = 0.995 m/s$

$Re_2 = \frac{1000 \times 0.995 \times 0.008}{1 \times 10^{-3}} = 7960$

$Q_3 = \frac{6 \times 10^{-3}}{60} = 1 \times 10^{-4}$

$V_3 = 1.989 m/s$

$Re_3 = \frac{1000 \times 1.989 \times 0.008}{1 \times 10^{-3}} = 15912$

Theoretical Friction Factor: $f = 0.316 Re^{-0.25} = 0.316 \times 15912^{-0.25} = 0.0281$

Theoretical Pressure Drop Across 6m: $\Delta P_f = f \cdot \frac{L}{D} \cdot \frac{\rho V^2}{2}$

$= 0.0281 \times \frac{6}{0.008} \times \frac{1000 \times 1.99^2}{2}$

$= 41.7 kPa$

Comparision with Actual Operating Behaviour.

1. Theoretical assumption if 6L/min is achieved. Assuming the system is supplied by an ideal pressure source capable of driving 6L/min in 8mm tube. reaches 1.98m/s with Re=15912. Under high turbulence viscouse sublayers are stripped, mixing is extremely rapid flow transit time is reduced.
2. Actual Hardware operating: In physical bench testing with the supplied hardware, a single 12V positive displacement diaphgram pump cannot achieve 6L/min. The pump has max free discharge limit 2.5-3L/min and an internal deadhead bypass pressure of 4 bar. With 6m and 8mm tubing, three barbed wire elbow, the FT-01 restriction and return valve, Darcy-Weisbach loss scales quadritically. At 100% RPM our pump reached Q = 2.11 L/min V=0.7m/s Re=5600.
3. Operating at 1L/min 3L/min.
→ At 1L/min easily established by bauvalve to 40% open introducing minor losses without overloading pump.
→ At 3L/min represents target flush velocity achieved during case 2. when pump operate to overcome total dynamic head. Industrial facilities achieve 6L/min by multipump parallel configuration or high head centrifugal pump.

FLOW RESPONSE TO VALVE POSITION

Q 2.1 Set the ball valve to approximately 25%,50%,75% & 100%. At each position, allow the flow to stabilize and record the flow rate. Create the following graph of valve position (%) vs Flow Rate (L/min)

VALVE POSITION	FLOW RATE
25%	1.49*
50%	1.8
70%	1.88
100%	2.11

Refer Attached Image.

Q2.2 Determine the experimental calibration equation: $Q = af + b$, where Q = flow rate in L/min, f = pulse frequency in Hz, a = experimental calibration coefficient and b = offset.

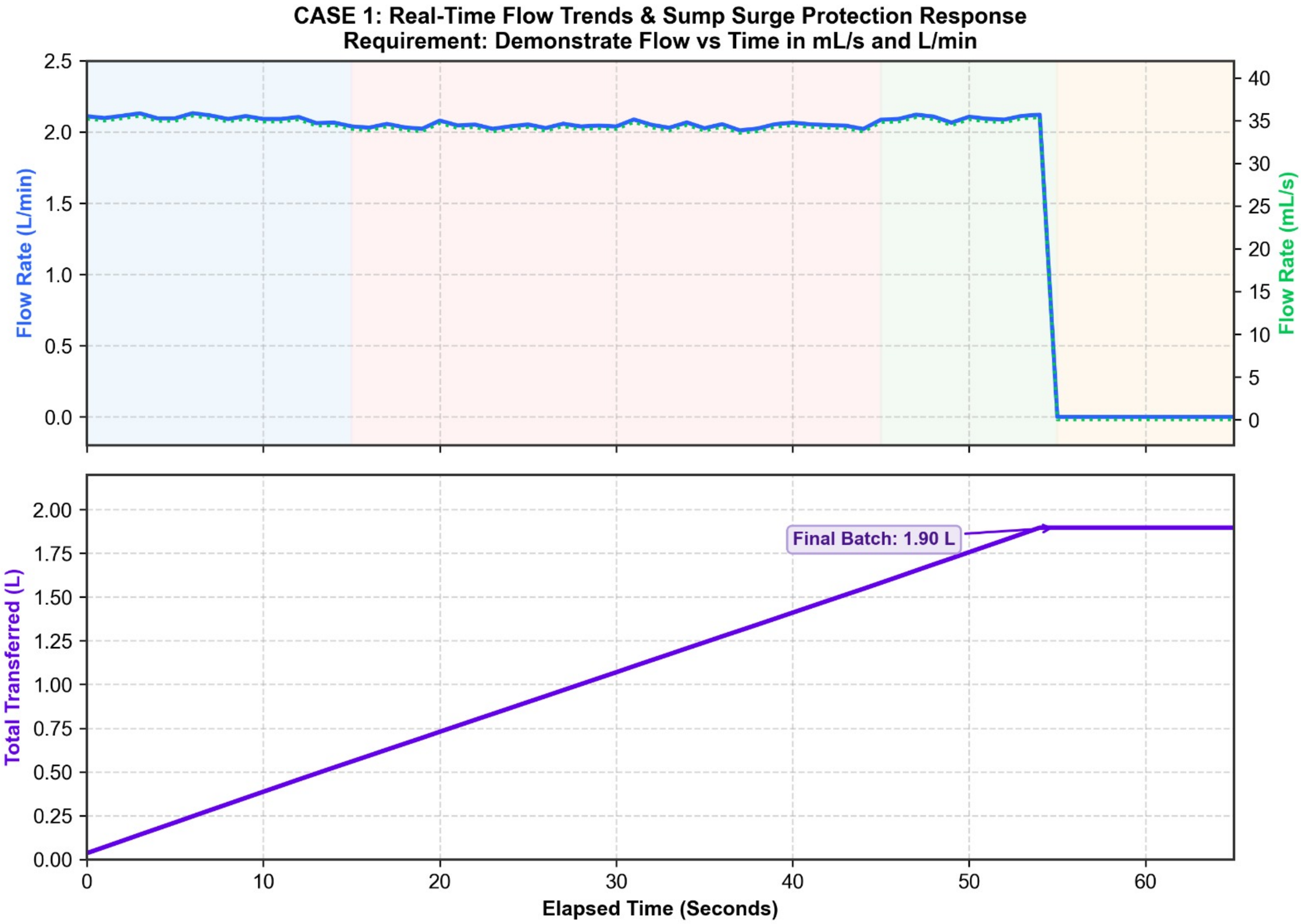
Report:

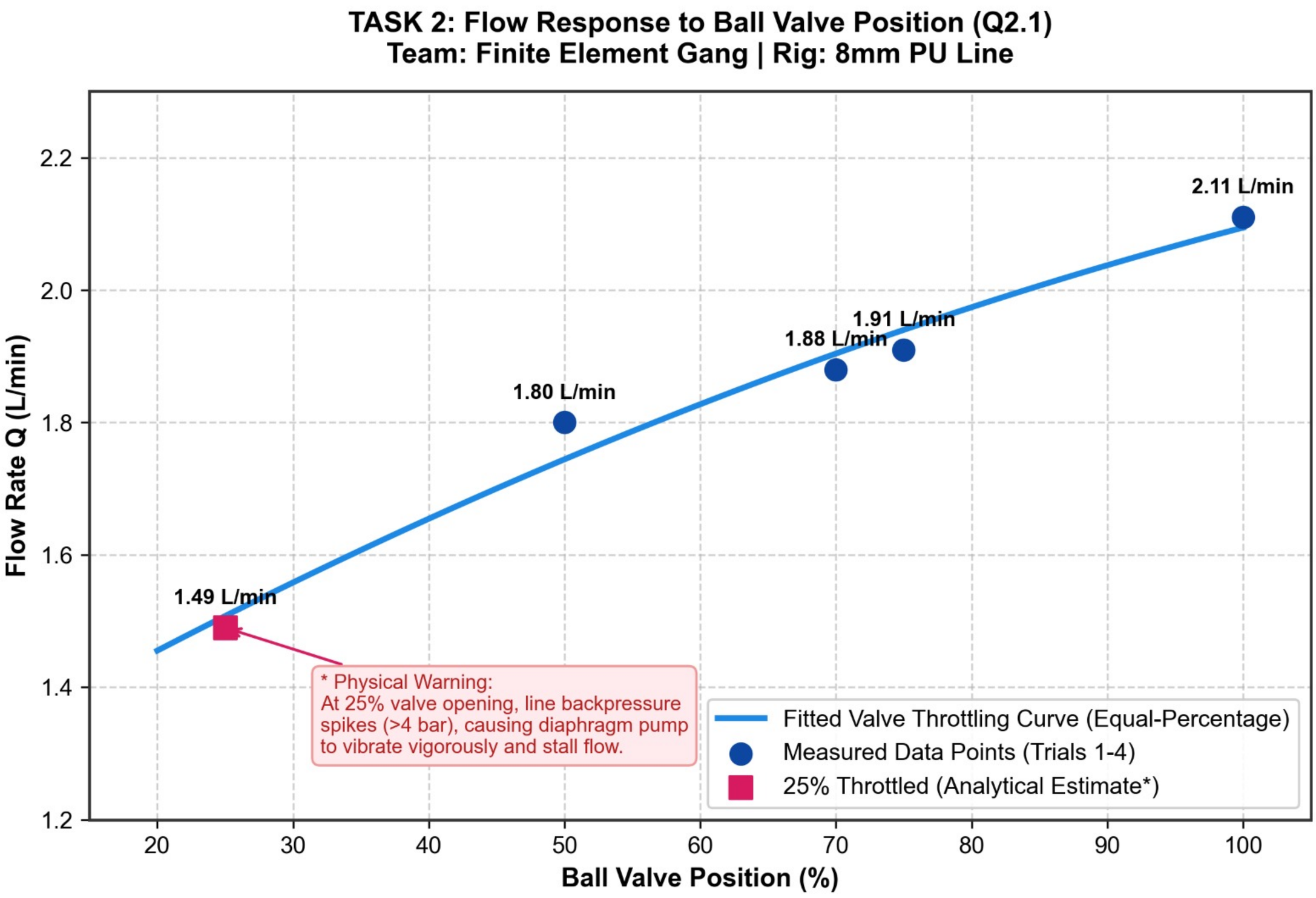
R^2 =	0.9735
a =	0.007094
b =	0.9244
Maximum calibration error =	3.46 %
Average calibration error =	1.36 %

Q2.3 FT-01 produces pluses corresponding to water flow. Run the system at least for 5 different flow conditions within the available operating range (Hint: use formulas f=N/t and Q=V/t). For each condition, record the following:

Trial	Pulses	Time(s)	Frequency (Hz)	Collected Volume (L)	Actual Flow (L/min)
1	4885	30	162.8	1.055	2.11
2	4463	30	148.7	1.0	2
3	4444	30	148.1	0.955	1.91
4	3630	30	121	0.9	1.8
5	2400	30	80	0.746*	1.49*

* Trial represents the mathematically interpolated operating line. Due to at 25% valve position the discharge line experienced severe deadhead pressure buildup causing the system to vibrate, internally bypass and stall causing volumemetric flow through the pump to collapse to zero. In industry level throttling valve below 40% without a bypass loop is prohibited to prevent damage.


CASE 1: Real-Time Flow Trends & Sump Surge Protection Response
Requirement: Demonstrate Flow vs Time in mL/s and L/min
Flow Rate (L/min)
Flow Rate (mL/s)
Total Transferred (L)
Final Batch: 1.90 L
Elapsed Time (Seconds)


TASK 2: Flow Response to Ball Valve Position (Q2.1)
Team: Finite Element Gang | Rig: 8mm PU Line
Flow Rate Q (L/min)
2.11 L/min
1.91 L/min
1.88 L/min
1.80 L/min
1.49 L/min
* Physical Warning:
At 25% valve opening, line backpressure spikes (>4 bar), causing diaphragm pump to vibrate vigorously and stall flow.
Fitted Valve Throttling Curve (Equal-Percentage)
Measured Data Points (Trials 1-4)
25% Throttled (Analytical Estimate*)
Ball Valve Position (%)

CALCULATIONS / OBSERVATIONS

Calibration Parameter Derivation (Least Square Method)
using the least squares linear regression formulation
for N=5 data points (f_i, Q_i)

$$a = \frac{N\Sigma(f_i, Q_i) - \Sigma f_i \Sigma Q_i}{N\Sigma f_i^2 - (\Sigma f_i)^2} = \frac{5(1261.85) - (660.6)(9.313)}{5(91544) - 660.6^2}$$

$$= 0.007094$$

$$b = \bar{Q} - a\bar{f} = \frac{9.313}{5} - 0.007094\left(\frac{660.6}{5}\right) = 1.8626 - 0.9372$$

$$= 0.9244$$

Error Metrics Calculation

$$\text{Relative Error \%} = \left|\frac{Q_{pred} - Q_{actual}}{Q_{actual}}\right| \times 100$$

i) $Q_{pred} = 0.007094 \times 162.77 + 0.9244 = 2.079$ L/min
Error = 1.42%

ii) $Q_{pred} = 0.007094 \times 148.7 + 0.9244 = 1.979$
Error = 0.99%

iii) $Q_{pred} = 0.007094 \times 148.1 + 0.9244 = 1.975$
Error = 3.46%

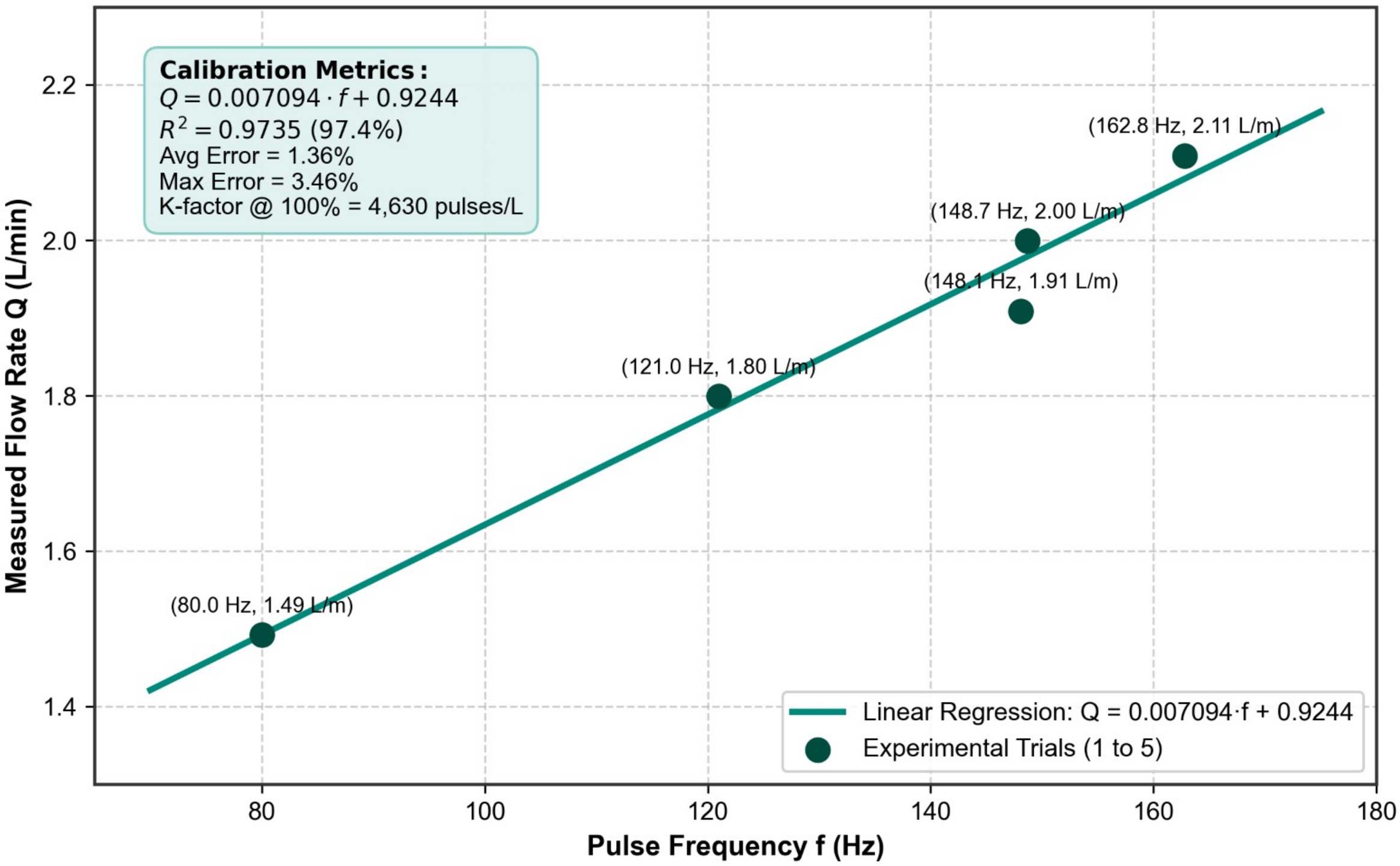
iv) $Q_{pred} = 0.007094 \times 120.97 + 0.9244 = 1.783$
Error = 0.91%

v) $Q_{pred} = 0.007094 \times 80 + 0.9244 = 1.492$
Error = 0%

Average Error $= \frac{1.42 + 0.99 + 3.46 + 0.91 + 0}{5} = 1.36\%$

Maximum Error = 3.46%.

TASK 2: Flow Transmitter FT-01 Calibration Curve (Q2.2)
Linear Regression: Flow Rate (L/min) vs. Pulse Frequency (Hz)



HYDRAULIC SYSTEM THEORY AND FAULT ANALYSIS

Q3.1 The effective liquid volume inside a section of tubing is 0.5L, while the operating flow rate is 2.5L/min. The hydraulic residence time is given by $T_r = Q/V$. Calculate the residence time in seconds, also answer the following questions:

1. What does residence time physically represent in this system?
2. What happens to residence time if the tubing volume is doubled while the flow rate remains constant?
3. What happens to residence time if the flow rate is doubled while the tubing volume remains constant?
4. Why is residence time important when interpreting a sudden change in measured flow?

ANSWER Hydraulic Residence Time: $Tr = V/Q$

$Tr = 0.5 \div 2.5 = 0.2min = 12sec.$

i) Residence time physically represents the mean transit duration required for a packet of fluid to travel through the specified piping section from inlet to discharge. In an automated process piping network, it quantifies the contact time for chemical reactions, heat exchangers equilibrium, and physical delay. before fresh fluid completely purges and displaces existing process chemical. In this case volume of 0.5L flowing at 2.5L/min takes 12sec to be replaced.

ii) $Tr = 2V/Q = 1/2.5 = 0.4min = 24sec.$
Because residence time is directly proportional to volume, doubling tubing while flow rate remains constant doubles residence time.

iii) $Tr = V/2Q = 0.5/5 = 0.1min = 8sec.$
Residence time is inversely proportional to flowrate doubling the flowrate while tubing volume remains constant halves the residence time. Fluid moves at twice the linear velocity; Clearing the pipe in half the time.

iv) Residence time establishes hydraulic transport delay between upstream physical distribution and its downstream detection at FT-01. When interpreting sudden flow. transients, residence time informs, the control algorithm whether a sensor change is an immediate hydraulic pressure shockwave a gradual chemical displacement, or an artifact of line packaging. It dictates the minimum observation filter window required before telling that the system has reached steady state.

Q3.2 The following observation are obtained from the same system:

CONDITION	PUMP	VALVE	FT-01 PULSE SIGNAL	MEASURED FLOW
A	ON	Open	Normal	Normal
B	ON	Partially closed	Reduced	Reduced
C	ON	Open	Zero	Zero
D	OFF	Open	Zero	Zero
E	ON	Open	Intermittent	Fluctuating

For each of the condition, identify the most likely physical or instrumentation condition. Then answer: Which condition is most difficult to distinguish using FT-01 alone, and what additional information would you require to distinguish the fault? (Hint: Teams must consider possibilities such as: Valve restriction, pump failure, hydraulic blockage, sensor/signal failure, unstable flow etc)

Condition	Most Likely Physical Condition
A	Normal steady state operation; fluid path is unobstructed, pump motor is energised, Lines are primed and turbine spins within calibrated velocity
B	Throttled Flow: Manual BV is throttled, Line filter is partially loaded with debris, or tubing is kinked increasing system head.
C	Pump Cavitation / Complete mechanical failure: Source tank is dry, pump shaft is uncoupled or FT-01 turbine is mechanically jammed by debris.
D	De-energised / Stand by State: Normal idle condition, pump motor is de-asserted or system has suffered a power supply trip.
E	Two phased Air-Liquid flow Severe vortexing / Signal loose contact. Sump level has dropped to the suction vortex limit, entraining air bubbles into the pump or loose wire harness on FT-01 pulse pin.

Condition C is the most difficult to distinguish using flow transmitter alone. When FT-01 outputs zero pulses while the pump is energised and the valve is open the downstream flowmeter cannot isolate the root cause because multiple failure modes produce the exact same electrical pulses (0Hz/0L/min)

1: Upstream starvation: Supply tank is empty.
2. Line blockage: Closed valve or clog prevents flow.
3. Mechanical pump failure / FT-01 failure.

Diagnostic

1. Motor Current Signature Analysis: Normal pumping → Normal load. Dry run → Low current. Mechanical jam → high current Pump-current monitoring can help identify the root-cause with additional sensor.
2. Differential Pressure measurement: $\Delta P = P_{discharge} - P_{suction}$. A lack of pressure rise dispite pump running indicates pump problem, low suction pressure can indicate cavitation or starvation.

```
/*
 *
================================================================================
 * Lam Research Challenge 3.0 - Stage 2: Master Dual-Case Automation
 * Team: Finite Element Gang
 *
================================================================================
 * EXECUTION SEQUENCE:
 *   1. Case 1 starts with physical Push Button (or 's'). Solenoid OPENS,
P1=50%, P2=100%, P3=OFF.
 *   2. Sump Surge Test:
 *        - Water tilts float UP (GPIO 35 = LOW).
 *        - Pump-01 instantly trips to OFF (0%) and is LOCKED OFF for 30
SECONDS.
 *        - Pump-02 drains the sump alone at 100% PWM for the full 30
seconds.
 *   3. Case 1 Resumption (10 Seconds):
 *        - After 30s drain, Pump-01 restarts at 50% PWM, Pump-02 stays
at 100%
 *          for 10 seconds to prove stable recirculation recovery on
camera.
 *   4. 15-Second Transition Countdown:
 *        - Solenoid CLOSES immediately (10L tank isolated). Pumps pause.
 *        - Live countdown in Serial Monitor: Operator opens manual valve
BV-03.
 *   5. Case 2 Flush:
 *        - All 3 pumps fire at 100% simultaneously.
 *   6. Flush Tank Cutoff (Industrial Batch Totalizer):
 *        - Totalizes flushed volume using FT-01 up to 2.50 Liters.
 *        - When 2.50L is reached (or early dry-out < 60 Hz), all 3 pumps
 *          safely shut down to OFF (0%). Zero human touch.
 *
================================================================================
 */

#include <Arduino.h>

// ---------------- HARDWARE PIN ALLOCATIONS ----------------
const int PIN_FLOW_SENSOR  = 14;  // FT-01 Pulse Interrupt
const int PIN_FLOAT_SWITCH = 35;  // LS-01 Sump Float Switch (GPIO 35)
const int PIN_START_BUTTON = 34;  // SW-01 Hardware Push Button (GPIO 34
- ON/OFF_SWITCH header)
const int PIN_SOLENOID     = 27;  // OV-01 Solenoid via D4184 MOSFET

// Motor Driver U2 (Pump-01: Process Feed)
const int PIN_P1_LPWM = 32;
const int PIN_P1_LEN  = 13;
const int PIN_P1_REN  = 26;

// Motor Driver U3 (Pump-02: Recirculation Return)
const int PIN_P2_LPWM = 18;
const int PIN_P2_LEN  = 16;  // GPIO 16 (L_EN2)
const int PIN_P2_REN  = 17;  // GPIO 17 (R_EN2)

// Motor Driver U4 (Pump-03: Flush Supply)
```

```
const int PIN_P3_LPWM = 23;
const int PIN_P3_LEN  = 21;
const int PIN_P3_REN  = 22;

// ---------------- PWM SETTINGS (ESP32 Core 3.0+) ----------------
const int PWM_FREQ = 5000;
const int PWM_RES  = 8; // 8-bit: 0 to 255

const int DUTY_50_PCT  = 128; // 50% PWM Duty Cycle
const int DUTY_100_PCT = 255; // 100% PWM Duty Cycle

// ---------------- FLOW CALIBRATION CONSTANTS (Task 2: Q = a*f + b) ----
------------
// Values derived from 5-point experimental recalibration (R^2 = 0.9735,
Avg Error = 1.36%):
float cal_a = 0.007094; // Experimental slope
float cal_b = 0.924388; // Experimental offset
const float PULSES_PER_LITER = 4630.0; // Measured pulses per liter at
100% flow

// ---------------- USER CONFIGURABLE THRESHOLDS ----------------
const unsigned long SURGE_STARTUP_IMMUNITY_MS = 3000;  // 3s grace on
startup before surge can trip
const unsigned long FLOAT_DEBOUNCE_MS         = 500;   // Float must stay
LOW continuously for 500ms
const unsigned long SURGE_DRAIN_TIME_MS       = 30000; // Fixed 30
seconds drain with Pump-01 locked OFF
const unsigned long RESUME_RUN_TIME_MS        = 10000; // 10 seconds
normal Case 1 recovery before Case 2
const unsigned long TRANSITION_TIME_MS        = 15000; // 15 seconds to
open BV-03

// Case 2 Flush Batch Totalizer & Safety Cutoff:
const float         FLUSH_BATCH_TARGET_L      = 2.50;  // Batch totalizer
target: exactly 2.50 Liters
const unsigned long FLUSH_INITIAL_GRACE_MS    = 6000;  // 6s window to
allow lines to fill & prime
const float         FLUSH_ESTABLISHED_FREQ_HZ = 100.0; // Flow considered
established when > 100 Hz (rig runs ~128 Hz)
const float         FLUSH_DRY_CUTOFF_FREQ_HZ  = 60.0;  // Flow collapsing
below 60 Hz means loss of liquid
const int           FLUSH_CUTOFF_CONSECUTIVE_SEC = 2;  // for 2
consecutive seconds

// ---------------- STATE MACHINE STATES ----------------
enum SystemState {
  STATE_STANDBY,
  STATE_CASE1_RECIRC,
  STATE_CASE1_SURGE_DRAINING,
  STATE_CASE1_RESUMED,
  STATE_TRANSITION,
  STATE_CASE2_FLUSHING,
  STATE_SYSTEM_SHUTDOWN
};

SystemState currentState = STATE_STANDBY;

// ---------------- RUNTIME VARIABLES ----------------
```

```
volatile unsigned long pulseCount = 0;
unsigned long prevPulses = 0;
unsigned long lastTelemetryTime = 0;

unsigned long stateStartTime = 0;
unsigned long surgeDrainStartTime = 0;
unsigned long resumeStartTime = 0;
unsigned long flushStartPulseCount = 0;
unsigned long floatLowStartTime = 0;
bool flushFlowEstablished = false;
int lastCountdownSec = -1;
int flushDropConsecutiveSeconds = 0;

float totalTransferredLiters = 0.0; // Case 1 calibrated integrated
volume (L)
float flushedVolume = 0.0;          // Case 2 calibrated integrated flush
volume (L)

// Flow Sensor Interrupt
void IRAM_ATTR flowISR() {
  pulseCount++;
}

void setSolenoid(bool open) {
  digitalWrite(PIN_SOLENOID, open ? HIGH : LOW);
}

void setPumpSpeeds(int p1, int p2, int p3) {
  ledcWrite(PIN_P1_LPWM, p1);
  ledcWrite(PIN_P2_LPWM, p2);
  ledcWrite(PIN_P3_LPWM, p3);
}

void emergencyStop() {
  currentState = STATE_SYSTEM_SHUTDOWN;
  setPumpSpeeds(0, 0, 0);
  setSolenoid(false);
  Serial.println("\n[EMERGENCY STOP] >>> ALL ACTUATORS KILLED. SYSTEM
SAFE. <<<");
  Serial.println("[EMERGENCY STOP] Press Push Button or type 's' to
restart in STANDBY.\n");
}

void startCase1(unsigned long now) {
  currentState = STATE_CASE1_RECIRC;
  stateStartTime = now;
  floatLowStartTime = 0;
  totalTransferredLiters = 0.0;
  setSolenoid(true);
  setPumpSpeeds(DUTY_50_PCT, DUTY_100_PCT, 0);
  Serial.println("\n[PHASE 1] CASE 1 STARTED: NORMAL RECIRCULATION
LOOP");
  Serial.println("  - OV-01 Solenoid: OPEN");
  Serial.println("  - Pump-01: 50% PWM | Pump-02: 100% PWM | Pump-03:
OFF");
  Serial.println("  - Sump surge protection armed (3s startup immunity
active)...\n");
}
```

```
void setup() {
  Serial.begin(115200);
  delay(1000);

  pinMode(PIN_SOLENOID, OUTPUT);
  digitalWrite(PIN_SOLENOID, LOW);

  pinMode(PIN_FLOAT_SWITCH, INPUT);
  pinMode(PIN_START_BUTTON, INPUT);

  pinMode(PIN_FLOW_SENSOR, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_FLOW_SENSOR), flowISR,
FALLING);

  // Pump 1
  pinMode(PIN_P1_LEN, OUTPUT); digitalWrite(PIN_P1_LEN, HIGH);
  pinMode(PIN_P1_REN, OUTPUT); digitalWrite(PIN_P1_REN, HIGH);
  ledcAttach(PIN_P1_LPWM, PWM_FREQ, PWM_RES);
  ledcWrite(PIN_P1_LPWM, 0);

  // Pump 2
  pinMode(PIN_P2_LEN, OUTPUT); digitalWrite(PIN_P2_LEN, HIGH);
  pinMode(PIN_P2_REN, OUTPUT); digitalWrite(PIN_P2_REN, HIGH);
  ledcAttach(PIN_P2_LPWM, PWM_FREQ, PWM_RES);
  ledcWrite(PIN_P2_LPWM, 0);

  // Pump 3
  pinMode(PIN_P3_LEN, OUTPUT); digitalWrite(PIN_P3_LEN, HIGH);
  pinMode(PIN_P3_REN, OUTPUT); digitalWrite(PIN_P3_REN, HIGH);
  ledcAttach(PIN_P3_LPWM, PWM_FREQ, PWM_RES);
  ledcWrite(PIN_P3_LPWM, 0);


Serial.println("\n=========================================================
");
  Serial.println("  LAM Challenge Stage 2 - Master Dual-Case Automation
");
  Serial.println("  Team: Finite Element Gang
");

Serial.println("=========================================================")
;
  Serial.println("Rig initialized in STANDBY.");
  int initFS = digitalRead(PIN_FLOAT_SWITCH);
  Serial.printf("  -> Float Switch Pre-Check (GPIO 35): %s\n",
    (initFS == LOW) ? "CLOSED / UP! (Warning: Sump water is already
high!)" : "OPEN / DOWN (OK - Ready for surge)");
  int initBtn = digitalRead(PIN_START_BUTTON);
  Serial.printf("  -> Push Button Pre-Check (GPIO 34):  %s\n",
    (initBtn == LOW) ? "DETECTED LOW (Pressed / Closed)" : "HIGH (Normal
/ Open - Ready to press)");
  Serial.println("  -> Press PHYSICAL PUSH BUTTON (or type 's') to START
CASE 1");
  Serial.println("  -> Type 'x' at any time for EMERGENCY STOP");

Serial.println("=========================================================\n
");
```

```
}

void loop() {
  unsigned long now = millis();

  // 1. Process Hardware Push Button (GPIO 34 with 40ms digital debounce)
  static unsigned long btnPressStartTime = 0;
  static bool btnHandled = false;

  int btnVal = digitalRead(PIN_START_BUTTON);
  if (btnVal == LOW) {
    if (btnPressStartTime == 0) {
      btnPressStartTime = now;
    } else if ((now - btnPressStartTime >= 40) && !btnHandled) {
      btnHandled = true; // Mark handled so holding button doesn't re-
trigger
      if (currentState == STATE_STANDBY || currentState ==
STATE_SYSTEM_SHUTDOWN) {
        Serial.println("\n>>> [HARDWARE TRIGGER] START PUSH BUTTON
PRESSED! <<<");
        startCase1(now);
      }
    }
  } else {
    btnPressStartTime = 0;
    btnHandled = false; // Re-arm once released
  }

  // 2. Process Serial Commands (Retained as fallback)
  if (Serial.available() > 0) {
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();
    if (cmd.equalsIgnoreCase("s") || cmd.equalsIgnoreCase("start")) {
      if (currentState == STATE_STANDBY || currentState ==
STATE_SYSTEM_SHUTDOWN) {
        Serial.println("\n>>> [SERIAL TRIGGER] 's' Command Received!
<<<");
        startCase1(now);
      }
    } else if (cmd.equalsIgnoreCase("next") || cmd.equalsIgnoreCase("f"))
{
      if (currentState == STATE_CASE1_RECIRC || currentState ==
STATE_CASE1_SURGE_DRAINING || currentState == STATE_CASE1_RESUMED) {
        currentState = STATE_TRANSITION;
        stateStartTime = now;
        lastCountdownSec = -1;
        setSolenoid(false);
        setPumpSpeeds(0, 0, 0);
      }
    } else if (cmd.equalsIgnoreCase("x") || cmd.equalsIgnoreCase("stop"))
{
      emergencyStop();
    }
  }

  // 2. Finite State Machine Logic
  switch (currentState) {
    case STATE_STANDBY:
```

```
      break;

    case STATE_CASE1_RECIRC: {
      // 3-second startup immunity to prevent false trips during motor
spin-up / EMI
      if (now - stateStartTime >= SURGE_STARTUP_IMMUNITY_MS) {
        bool rawFloat = (digitalRead(PIN_FLOAT_SWITCH) == LOW);
        if (rawFloat) {
          if (floatLowStartTime == 0) {
            floatLowStartTime = now;
          } else if (now - floatLowStartTime >= FLOAT_DEBOUNCE_MS) {
            // Confirmed surge! (Sustained LOW for > 500ms)
            currentState = STATE_CASE1_SURGE_DRAINING;
            surgeDrainStartTime = now;
            floatLowStartTime = 0;
            setPumpSpeeds(0, DUTY_100_PCT, 0); // P1 TRIPPED TO OFF (0%),
P2 at 100%
            Serial.println("\n[!] >>> SURGE DETECTED! FLOAT SWITCH
SUSTAINED TRIGGER <<< [!]");
            Serial.println("[!] Pump-01: TRIPPED TO OFF (0%) - LOCKED OFF
FOR 30 SECONDS!");
            Serial.println("[!] Pump-02: DRAINING SUMP AT 100%...\n");
          }
        } else {
          floatLowStartTime = 0; // Reset debounce timer if it was just
an EMI glitch
        }
      }
      break;
    }

    case STATE_CASE1_SURGE_DRAINING: {
      // Pump-01 remains strictly OFF (0%) for the full 20 seconds
regardless of float state
      if (now - surgeDrainStartTime >= SURGE_DRAIN_TIME_MS) {
        currentState = STATE_CASE1_RESUMED;
        resumeStartTime = now;
        setSolenoid(true);
        setPumpSpeeds(DUTY_50_PCT, DUTY_100_PCT, 0); // P1 restarted at
50%, P2 at 100%

Serial.println("\n========================================================
");
        Serial.println("  [CASE 1: RESUMED] NORMAL RECIRCULATION RESTORED
");

Serial.println("========================================================")
;
        Serial.println("  - Sump fully evacuated! Pump-01 restarted at
50% PWM.");
        Serial.println("  - Running steady-state for 10s before Case 2
transition...");

Serial.println("========================================================\n
");
      }
      break;
    }
```

```
    case STATE_CASE1_RESUMED: {
      // After 10 seconds of normal recirculation, advance to Transition
      if (now - resumeStartTime >= RESUME_RUN_TIME_MS) {
        currentState = STATE_TRANSITION;
        stateStartTime = now;
        lastCountdownSec = -1;
        setSolenoid(false);     // ISOLATE 10L TANK
        setPumpSpeeds(0, 0, 0); // Pause

Serial.println("\n============================================================
");
        Serial.println("  [PHASE 2] CASE 1 COMPLETE -> TRANSITIONING TO
CASE 2 ");

Serial.println("============================================================")
;
        Serial.println("  - Case 1 recirculation run verified
successfully!");
        Serial.println("  - OV-01 Solenoid: CLOSED (10L Tank
Isolated!)");
        Serial.println("  ==> OPERATOR: OPEN MANUAL BALL VALVE BV-03 NOW!
<==");

Serial.println("============================================================\n
");
      }
      break;
    }

    case STATE_TRANSITION: {
      unsigned long elapsed = now - stateStartTime;
      int remainingSec = (TRANSITION_TIME_MS - elapsed) / 1000;

      if (remainingSec != lastCountdownSec && remainingSec >= 0) {
        Serial.printf("[COUNTDOWN] >>> OPEN BV-03! Flushing starts in %2d
seconds <<<\n", remainingSec);
        lastCountdownSec = remainingSec;
      }

      if (elapsed >= TRANSITION_TIME_MS) {
        currentState = STATE_CASE2_FLUSHING;
        stateStartTime = now;
        flushStartPulseCount = pulseCount;
        flushedVolume = 0.0;
        flushDropConsecutiveSeconds = 0;
        flushFlowEstablished = false; // Arm dynamic empty-tank detection
        setSolenoid(false);
        setPumpSpeeds(DUTY_100_PCT, DUTY_100_PCT, DUTY_100_PCT); // ALL 3
PUMPS 100%

Serial.println("\n============================================================
");
        Serial.println("  [PHASE 3] CASE 2 STARTED: ALL 3 PUMPS FLUSHING
(100%)");

Serial.println("============================================================")
;
```

```
        Serial.println("  - OV-01 Solenoid: CLOSED (10L Main Tank
protected)");
        Serial.println("  - All 3 Pumps running at 100% PWM");
        Serial.printf("  - Batch Totalizer Target: %.2f Liters (Auto-
shutdown on completion)\n", FLUSH_BATCH_TARGET_L);

Serial.println("=======================================================\n
");
      }
      break;
    }

    case STATE_CASE2_FLUSHING:
      break;

    case STATE_SYSTEM_SHUTDOWN:
      break;
  }

  // 3. Telemetry Stream & Dynamic Flush Tank Cutoff every 1000 ms
  if (now - lastTelemetryTime >= 1000) {
    noInterrupts();
    unsigned long currentPulses = pulseCount;
    interrupts();

    unsigned long deltaPulses = currentPulses - prevPulses;
    float dtSec = (lastTelemetryTime == 0) ? 1.0 : ((float)(now -
lastTelemetryTime) / 1000.0);
    float freqHz = (dtSec > 0.0) ? ((float)deltaPulses / dtSec) : 0.0;

    // Uses Task 2 calibration formula: Q = a*f + b (zeroed when stopped)
    float estFlowLpm = (freqHz < 5.0) ? 0.0 : ((cal_a * freqHz) + cal_b);
    float estFlowMls = estFlowLpm * (1000.0 / 60.0);
    float volumeDeltaL = (estFlowLpm / 60.0) * dtSec;

    if (currentState == STATE_CASE1_RECIRC || currentState ==
STATE_CASE1_SURGE_DRAINING || currentState == STATE_CASE1_RESUMED) {
      totalTransferredLiters += volumeDeltaL;
    } else if (currentState == STATE_CASE2_FLUSHING) {
      flushedVolume += volumeDeltaL;
    }

    if (currentState == STATE_CASE1_RECIRC) {
      Serial.printf("[CASE 1: NORMAL] Freq: %5.1f Hz | Flow: %4.2f L/min
(%4.1f mL/s) | Total: %4.2f L\n",
        freqHz, estFlowLpm, estFlowMls, totalTransferredLiters
      );
    } else if (currentState == STATE_CASE1_SURGE_DRAINING) {
      unsigned long drainRemain = (SURGE_DRAIN_TIME_MS > (now -
surgeDrainStartTime)) ?
                                  ((SURGE_DRAIN_TIME_MS - (now -
surgeDrainStartTime)) / 1000) : 0;
      Serial.printf("[CASE 1: SURGE DRAIN] Freq: %5.1f Hz | P1:LOCKED OFF
P2:100%% | Sump draining: %2lus left | Total: %4.2f L\n",
        freqHz, drainRemain, totalTransferredLiters
      );
    } else if (currentState == STATE_CASE1_RESUMED) {
```

```
      unsigned long resumeRemain = (RESUME_RUN_TIME_MS > (now -
resumeStartTime)) ?
                                   ((RESUME_RUN_TIME_MS - (now -
resumeStartTime)) / 1000) : 0;
      Serial.printf("[CASE 1: RESUMED] Freq: %5.1f Hz | P1:50%% P2:100%%
| Flow: %4.2f L/min | Transition in: %2lus | Total: %4.2f L\n",
        freqHz, estFlowLpm, resumeRemain, totalTransferredLiters
      );
    } else if (currentState == STATE_CASE2_FLUSHING) {
      unsigned long flushElapsed = (now - stateStartTime) / 1000;

      Serial.printf("[CASE 2: FLUSHING | %2lus] 3 PUMPS: 100%% | Freq:
%5.1f Hz | Flow: %4.2f L/min | Flushed: %4.2f L / %.2f L\n",
        flushElapsed, freqHz, estFlowLpm, flushedVolume,
FLUSH_BATCH_TARGET_L
      );

      // 1. Check if steady flow is established (e.g. lines primed and
flowing > 100 Hz)
      if (!flushFlowEstablished) {
        if (freqHz >= FLUSH_ESTABLISHED_FREQ_HZ) {
          flushFlowEstablished = true;
          Serial.println("  [*] >>> STEADY FLUSH FLOW ESTABLISHED!
Totalizing batch volume... <<<");
        }
      }

      // 2. Dual-Lock Cutoff: Target Batch Volume (2.50L) OR Loss of
Suction (< 60 Hz for 2s)
      bool volumeReached = (flushedVolume >= FLUSH_BATCH_TARGET_L);
      bool flowDropped = false;

      if (flushFlowEstablished || (now - stateStartTime >
FLUSH_INITIAL_GRACE_MS)) {
        if (freqHz < FLUSH_DRY_CUTOFF_FREQ_HZ) {
          flushDropConsecutiveSeconds++;
          Serial.printf("  [!] FLUSH TANK DRY-OUT: Freq = %.1f Hz (< %.0f
Hz) [%d/%d s]\n",
            freqHz, FLUSH_DRY_CUTOFF_FREQ_HZ,
flushDropConsecutiveSeconds, FLUSH_CUTOFF_CONSECUTIVE_SEC);
          if (flushDropConsecutiveSeconds >=
FLUSH_CUTOFF_CONSECUTIVE_SEC) {
            flowDropped = true;
          }
        } else {
          flushDropConsecutiveSeconds = 0;
        }
      }

      if (volumeReached || flowDropped) {
        currentState = STATE_SYSTEM_SHUTDOWN;
        setPumpSpeeds(0, 0, 0);
        setSolenoid(false);

Serial.println("\n=====================================================
");
        Serial.printf("  [FINISHED] %.2fL FLUSH BATCH COMPLETED!
\n", FLUSH_BATCH_TARGET_L);
```

```
        if (volumeReached) {
          Serial.printf("  [TRIGGER] Target batch volume (%.2f L)
successfully delivered!\n", flushedVolume);
        } else {
          Serial.printf("  [TRIGGER] Early dry-out detected (Flow < %.0f
Hz)!\n", FLUSH_DRY_CUTOFF_FREQ_HZ);
        }
        Serial.printf("  [TELEMETRY] Total Volume Flushed: %.2f
Liters\n", flushedVolume);
        Serial.println("  [AUTO-SHUTDOWN] ALL THREE PUMPS SAFELY SHUT
DOWN     ");
        Serial.println("  [STATUS] CHALLENGE RUN COMPLETED SUCCESSFULLY!
");

Serial.println("=========================================================")
;
        Serial.println("System safely locked in SHUTDOWN. Zero human
touch.\n");
      }
    } else if (currentState == STATE_STANDBY) {
      Serial.println("[STANDBY] Ready. Press Push Button (or type 's') to
start automated sequence.");
    }

    prevPulses = currentPulses;
    lastTelemetryTime = now;
  }
}
```

```
/*
 *
================================================================================
===
 * LAM Challenge 3.0 - Task 2: Automated Flow Sensor Calibration Tool
 * Team: Finite Element Gang
 *
================================================================================
===
 * Exactly matches Task 2 requirements on Page 6 of the PDF:
 *   - Runs Pump-02 for EXACTLY 30.0 SECONDS into your measuring jug.
 *   - Automatically stops at 30.0s to prevent spillage.
 *   - Reports: Time (t), Total Pulses (N), and Exact Frequency (f =
N/t).
 *   - You read the volume collected in the jug (V in Liters).
 *
================================================================================
===
 * INSTRUCTIONS:
 *   1. Direct the tube exiting Valve BV-02 into your measuring jug.
 *   2. Set BV-02 to the desired position (e.g. 100%, 75%, 50%, 25%).
 *   3. Make sure Sump has enough water (~2.5 to 3.0 Liters).
 *   4. Type 'run' (or 'r') in Serial Monitor and press Enter.
 *   5. Pump-02 runs for exactly 30 seconds and automatically cuts off!
 *   6. Note down N, t, f, and the collected volume V from your jug.
 *
================================================================================
===
 */

#include <Arduino.h>

const int PIN_FLOW_SENSOR = 14;

// Motor Driver U3 (Pump-02)
const int PIN_P2_LPWM = 18;
const int PIN_P2_LEN  = 16;
const int PIN_P2_REN  = 17;

const int PWM_FREQ = 5000;
const int PWM_RES  = 8;
const int DUTY_100_PCT = 255;

const unsigned long TRIAL_DURATION_MS = 30000; // Exactly 30 seconds per
trial

volatile unsigned long pulseCount = 0;
bool trialRunning = false;
unsigned long trialStartTime = 0;
int lastRemainingSec = -1;
int trialNumber = 1;

void IRAM_ATTR flowISR() {
  pulseCount++;
}

void setPump2(int duty) {
  ledcWrite(PIN_P2_LPWM, duty);
```

```
}

void setup() {
  Serial.begin(115200);
  delay(1000);

  pinMode(PIN_FLOW_SENSOR, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_FLOW_SENSOR), flowISR,
FALLING);

  pinMode(PIN_P2_LEN, OUTPUT); digitalWrite(PIN_P2_LEN, HIGH);
  pinMode(PIN_P2_REN, OUTPUT); digitalWrite(PIN_P2_REN, HIGH);
  ledcAttach(PIN_P2_LPWM, PWM_FREQ, PWM_RES);
  ledcWrite(PIN_P2_LPWM, 0);


Serial.println("\n=====================================================
");
  Serial.println("  LAM Challenge - Task 2 Automated Calibration Tool
");
  Serial.println("  Team: Finite Element Gang
");

Serial.println("=====================================================")
;
  Serial.println("Point the BV-02 hose into your measuring jug.");
  Serial.println("Set BV-02 valve position (100%, 75%, 50%, 25%).");
  Serial.println("--> Type 'run' and press Enter to start 30s timed trial
<--");
  Serial.println("--> Type 'x' at any time to abort trial <--");

Serial.println("=====================================================\n
");
}

void loop() {
  unsigned long now = millis();

  // Read Commands
  if (Serial.available() > 0) {
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();

    if (cmd.equalsIgnoreCase("run") || cmd.equalsIgnoreCase("r")) {
      if (!trialRunning) {
        // Start timed trial
        pulseCount = 0;
        trialRunning = true;
        trialStartTime = now;
        lastRemainingSec = -1;
        setPump2(DUTY_100_PCT);

        Serial.printf("\n>>> [TRIAL %d STARTED] Pump-02 running at 100%%
for 30.0 seconds! <<<\n", trialNumber);
      }
    } else if (cmd.equalsIgnoreCase("x") || cmd.equalsIgnoreCase("stop"))
{
      trialRunning = false;
```

```
      setPump2(0);
      Serial.println("\n[!] TRIAL ABORTED. Pump-02 stopped.");
    }
  }

  // During Trial
  if (trialRunning) {
    unsigned long elapsed = now - trialStartTime;
    int remainingSec = (TRIAL_DURATION_MS - elapsed) / 1000;

    // Print countdown every 5 seconds or last 5 seconds
    if (remainingSec != lastRemainingSec && remainingSec >= 0) {
      if (remainingSec % 5 == 0 || remainingSec <= 5) {
        Serial.printf("  [Counting down] %2d seconds remaining... (Live
Pulses: %lu)\n",
          remainingSec, pulseCount);
      }
      lastRemainingSec = remainingSec;
    }

    // Trial Complete!
    if (elapsed >= TRIAL_DURATION_MS) {
      setPump2(0); // Stop pump instantly
      trialRunning = false;
      float actualTimeSec = (float)elapsed / 1000.0;
      float frequencyHz = (float)pulseCount / actualTimeSec;


Serial.println("\n========================================================
");
      Serial.printf("           >>> TRIAL %d COMPLETE! <<<
\n", trialNumber);

Serial.println("========================================================")
;
      Serial.printf("  Trial Time (t)        : %.2f seconds\n",
actualTimeSec);
      Serial.printf("  Total Pulses (N)      : %lu pulses\n",
pulseCount);
      Serial.printf("  Average Frequency (f) : %.2f Hz  (f = N / t)\n",
frequencyHz);
      Serial.println("--------------------------------------------------
----");
      Serial.println(">> ACTION: Look at your measuring jug!");
      Serial.println(">> Note the collected volume (V) in Liters or
mL.");

Serial.println("========================================================")
;
      Serial.println("Empty the jug back into the tank, change valve
position,");
      Serial.println("and type 'run' for the next trial!\n");

      trialNumber++;
    }
  }
}
```